Защита: вероятные вопросы и обоснованные ответы

Английский оригинал: DEFENSE_FAQ.md.

Двадцать восемь вопросов, которые рецензент, вероятно, задаст, с ответом и файлом, командой или числом, которые его подкрепляют. Здесь ничего не утверждается без источника. Двадцать восемь были перепроверены по репозиторию на финальной ревизии; каждый путь и имя теста ниже разрешаются, а числа совпадают с docs/METRICS.md. Более короткий список для судьи, которому сначала нужна суть, — docs/FAQ_FOR_JUDGES.md.

1. Как вы отделяете наблюдение от гипотезы?

Наблюдение — диапазон байт, который прочитал движок, с его пакетом, номером последовательности и временем. Гипотеза — поле, которое правило помечает hypothesis: true, то есть смысл байтов предполагается. Движок правил сообщает значение как факт (значение плюс провенанс); движок альтернатив сообщает смысл как кандидата с баллом. Два пути никогда не делят код. Доказательства: src/protocol/engine.py, src/hypothesis/alternatives.py, REPORT.md раздел 3.

2. Почему фрейминг — с префиксом длины и length_covers: payload?

Потому что это единственный из трёх кандидатов, который фреймит каждый чистый поток точно, без остаточных байтов и без обрезанного сообщения. payload_and_length_field потребляет поток, но оставляет невозможные байты команды; entire_message останавливается после первого размера. Доказательство: перепись фрейминга в docs/CORPUS_ANALYSIS.md раздел 2.

3. Почему вы расширили набор команд с [1] до [1, 2, 3]?

Потому что v1 дало восемьдесят контрпримеров, все «значение 2 или 3 не в ожидаемом [1]», а движок альтернатив говорит, что поле — небольшой замкнутый enum (entropy_enum: поддержка 140, противоречий 0), тогда как constant (чтение v1) опровергается 80 из 140. Доказательства: docs/CORPUS_ANALYSIS.md разделы 4–5, docs/RULE_V1_V2.md раздел 3.

4. Почему команда остаётся гипотезой после проверки?

Проверка на корпусе подтверждает согласие, а не смысл. 0x01/0x02/0x03 подходят под код команды, но корпус синтетический и небольшой, поэтому правило сохраняет флаг. Поле подтверждено в области, а не доказано. Доказательства: REPORT.md раздел 3, src/hypothesis/status.py.

5. Что именно такое контрпример?

mismatched-сообщение, привязанное к своим байтам и исходному пакету: session_id, direction, message_offset, message_length, bytes_hex, reason. Это не отброшенная строка; он хранится в VerificationReport.contradictions и показывается в GUI. Пример: 02000003130055 в s2 A_to_B, причина «значение 2 не в ожидаемом [1]». Доказательства: src/hypothesis/corpus.py, docs/CORPUS_ANALYSIS.md раздел 4.

6. Как определяются точность и покрытие?

Точность — matched / (matched + mismatched), доля сообщений, о которых правило говорило, в которых оно оказалось право. Покрытие считает, какую долю корпуса правило вообще затрагивает; потоки вне области — `not_applicable`, а не молча считаются matched. Для v1 на `corpus_capture_01`: точность 60/140 = 0.43; v2: 140/140 = 1.00. Доказательства: `src/hypothesis/metrics.py`, `docs/CORPUS_ANALYSIS.md` раздел 6.

7. Почему `B_to_A` сообщается как not_applicable?

Правило ограничено `A_to_B`; оно не делает утверждений о другом направлении. Движок записывает это как `not_applicable`, а не выдумывает совпадение. Доказательство: `tests/protocol/test_engine.py::test_scoped_rule_marks_other_direction_not_applicable`.

8. Что происходит с результатом, произведённым более старой версией правила?

Он помечается `outdated`, с причиной `superseded_by_newer_rule`. Он остаётся на диске со своей версией, а сравнение версий — единственное место, где числа v1 и v2 стоят рядом. Старый результат никогда не показывается как текущий. Доказательства: `src/hypothesis/versioning.py`, `src/ui/model.py::_mark_outdated`, `tests/hypothesis/test_versioning.py`.

9. Почему пропуски не заполняются нулями?

Заполнение нулями выдумало бы байты, которых никогда не наблюдали, и могло бы превратить неполное сообщение в якобы корректное. Пропуск становится диагностикой `gap`; сообщение над ним — `incomplete`. Доказательства: `tests/protocol/test_engine.py::test_gap_in_field_range_is_incomplete`, `docs/CORPUS_ANALYSIS.md` раздел 1.

10. А что насчёт неоднозначного перекрытия?

Конфликтующая ретрансмиссия сохраняет первые байты и записывает диагностику `ambiguity` с обеими версиями. Поле над неоднозначностью — `ambiguous`, а не `matched`. Доказательство: `tests/protocol/test_engine.py::test_ambiguity_in_field_range_is_ambiguous`.

11. Где правило останавливается?

На `synthetic_live`. v2 не совпадает там ни с чем. Захват использует little-endian длину, поэтому при big-endian длине правила оба потока обрезаны: ни один не фреймится чисто, и ограниченное правило сообщает одно `incomplete`-сообщение, а остальные `not_applicable`. Правило падает на поле длины прежде, чем судится любой байт команды, и говорит об этом, а не гадает. Доказательства: `docs/CORPUS_ANALYSIS.md` раздел 7, `tests/integration/test_reference_report.py::test_v2_fails_at_the_synthetic_boundary`.

12. «Совпадает везде на корпусе» — то же, что доказательство?

Нет. Это значит, что правило подтверждено внутри проверенных байтов. Весь смысл движка альтернатив и граничного захвата — держать это различие видимым. Доказательства: `REPORT.md` раздел 3, раздел 8.

13. Почему альтернативные чтения берутся из фиксированного набора?

Движок перечисляет именованных кандидатов: `constant`, `counter`, `message_length`, `remaining_length`, `low_cardinality`, `alias_of_<field>`, полосы энтропии (`entropy_enum`, `entropy_parameter`, `entropy_random`), `periodicity`, `bit_pattern_<width>`, `flags_high`, `delta_correlation`, `delta_encoding`, `endianness`, `offset_shift_+1/offset_shift_-1`, `xor_mask_<mask>`, `checksum_<algorithm>_<region>` и `journal_<key>`. Смысл вне набора предложен не будет. Это заявлено как ограничение на презентации и здесь. Доказательства: `src/hypothesis/alternatives.py`, `docs/PRESENTATION_VERIFICATION.md`, слайд ограничений.

14. Откуда вы знаете, что корпусное правило не переобучено под один захват?

v2 проверено на втором, независимом захвате (`corpus_capture_02`) без единого контрпримера: 40 matched, 0 mismatched. Доказательство: `tests/integration/test_reference_report.py::test_v2_transfers_to_the_second_capture_without_counterexamples`.

15. Почему `rule_id` отделён от `rule_version`?

`rule_id` называет правило на протяжении его истории; `rule_version` считает изменения. Перезагрузка идентичного текста — no-op, поэтому повторный прогон не помечает собственный результат outdated, а любое изменение поднимает версию. Доказательства: `src/protocol/rule.py`, `src/ui/model.py::RuleModel.reload`.

16. Как проверяется правило?

По `docs/schemas/rule.schema.json` (JSON Schema черновика 2020-12) через `jsonschema.Draft202012Validator` и парсером в `src/protocol/rule.py`. Неверный фрейминг или плохое смещение поля отвергается прежде, чем правило запустится. Доказательство: `tests/protocol/test_rule.py`.

17. Почему не PyShark или Kaitai для самого чтения?

PyShark нужен `tshark`; инструмент должен работать только с зависимостями Python. dpkt читает и pcap, и pcapng без внешнего бинарника. Kaitai предлагается как выгрузка готового правила, а не как движок. Доказательства: `ARCHITECTURE.md`, `src/protocol/export.py`.

18. Как инструмент ведёт себя без корпуса?

Дымовые тесты корпуса пропускаются, а не подделывают результат, а CLI падает с ясной ошибкой, если файл отсутствует. Ничего не выдумывается, когда данных нет. Доказательства: `tests/integration/test_reference_report.py` (маркер пропуска), `docs/demo_cli.md`.

19. Почему у байта флагов не указан смысл?

Он константен (`0x00`) в каждом наблюдаемом сообщении, поэтому ни одно чтение нельзя отличить от любого другого; правило объявляет `expected: [0]` и ничего больше. Смысл был бы догадкой без доказательств. Доказательства: `docs/CORPUS_ANALYSIS.md` раздел 3, `docs/PRESENTATION_VERIFICATION.md`, слайд ограничений.

20. Что бы вы сделали дальше?

Три конкретных шага, каждый нацелен на ограничение выше: захватить трафик настоящего устройства, чтобы проверить оговорку о синтетическом корпусе; разобрать ответы `B_to_A` по полям, чтобы правило покрыло оба направления; и провести параметр через сессии, чтобы увидеть, меняется ли когда-нибудь байт флагов, что позволило бы отличить второе чтение. Доказательства: `REPORT.md` раздел 12, слайд ограничений презентации.

21. Почему не использовать Wireshark как бэкенд вместо dpkt?

Wireshark — программа, а не библиотека; её использование означало бы вызов `tshark`, так что инструмент зависел бы от внешнего бинарника и его версии. dpkt читает и pcap, и pcapng на чистом Python без внешнего процесса, что сохраняет проект воспроизводимым из одного `pip install`. Модуль захвата — тонкий слой над dpkt и тестируется на обоих контейнерных форматах. Доказательства: `ARCHITECTURE.md` (таблица зависимостей), `src/capture/`.

22. Откуда вы знаете, что правило не переобучено под корпус?

Переобучение проверяется переносом, а не самой подгонкой. Правило v2 было подогнано на `corpus_capture_01`, а затем применено без изменений к `corpus_capture_02`, отдельной записи: 40 matched, 0 контрпримеров. Оно также было применено к `synthetic_live`, где не совпадает ни с чем; правило, настроенное под корпус, не имело бы причин падать там, а инструмент записывает этот сбой вместо того, чтобы его скрыть. Доказательства: `REPORT.md` разделы 7–8, `tests/integration/test_reference_report.py::test_v2_transfers_to_the_second_capture_without_counterexamples`.

23. Корпус мал; это проблема?

Корпус держит 185 транзакций журнала и 369 сфреймленных сообщений по трём захватам (280 в `corpus_capture_01`, 80 в `corpus_capture_02`, 9 в захвате дефектов). Этого достаточно, чтобы показать, что фрейминг исчерпывающий, а набор команд замкнут на наблюдаемых значениях, но недостаточно, чтобы доказать общую грамматику. Утверждение держится в пределах того, что поддерживают байты: поле подтверждено в области, а правило сохраняет `hypothesis: true` там, где смысл выведен. Доказательства: `REPORT.md` раздел 3, раздел 12, `docs/CORPUS_ANALYSIS.md` раздел 8.

24. Как вы проверяете, что правило не подогнано под один файл?

Двумя способами. Во-первых, перенос: то же правило гоняется на втором захвате, под который оно не подгонялось, и счётчики проверяются там. Во-вторых, контрпримеры: v1 сохраняется со своими восемьюдесятью противоречиями, поэтому граница правила видна как данные, а не заявлена. Правило, которое работало бы только на файле, под который написано, показало бы ноль контрпримеров на этом файле и стену их на следующем, что и демонстрирует граничный захват. Доказательства: `REPORT.md` разделы 5, 7, 8.

25. Где хранятся наблюдения исследователя?

Наблюдения и гипотезы живут в JSON правила и в структурированном отчёте об исследовании. Каждое поле несёт своё смещение, тип и флаг `hypothesis`, а каждый вердикт несёт диапазон байт и пакет, из которого пришёл, поэтому рассуждение восстановимо из одних документов. Отдельная таблица `annotations.sqlite` не реализована; это осознанный выбор в пользу самодо-

статочного формата JSON, который переносится с проектом и не добавляет зависимости. Условие задания не требует конкретного механизма хранения. Доказательства: `docs/schemas/rule.schema.json`, `REPORT.md`, `src/hypothesis/corpus.py`.

26. Как проверка воспроизведётся через год на другой машине?

Каждое число производится из поставляемых файлов командой, а окружение закреплено: Python 3.12 или новее и зависимости в `pyproject.toml`. Отчёт указывает, как генерируется каждое значение, поэтому его можно пересобрать, а не доверять ему. Если файл корпуса отсутствует, дымовые тесты пропускаются, а CLI падает с ясной ошибкой вместо выдумывания результата. Доказательства: `docs/PORTABILITY.md`, `docs/demo_cli.md`, `pyproject.toml`.

27. Откуда вы знаете, что `length_covers: payload` — верная семантика?

Это не утверждается; это единственный из трёх кандидатов, который фреймит каждый чистый поток без остаточных байтов и без обрезанного сообщения. `payload_and_length_field` потребляет поток, но оставляет байты команды на невозможных значениях, а `entire_message` останавливается после первого размера. Решение — измерение на корпусе, закреплённое в обеих версиях правила как `"length_covers": "payload"`. Доказательства: `docs/CORPUS_ANALYSIS.md` раздел 2, `REPORT.md` раздел 4.

28. Зачем вообще GUI, если всё работает из CLI?

CLI — основной, скриптуемый путь: apply, verify, diff, metrics, correlate и export все работают без окна. Окно существует для того, в чём терминал плох: видеть байты, провенанс и контрпримеры вместе и двигаться между сообщением, его пакетом и его вердиктом. Оба пути вызывают один движок, поэтому окно никогда не вычисляет второй ответ. Доказательства: `docs/demo.md` (проход по окну), `docs/demo_cli.md` (то же исследование без окна), `src/ui/`.

Откуда берутся числа

```
python -m pytest tests/integration/test_reference_report.py -q
python -m src.protocol.cli verify --rule examples/corpus_rule_v1.json \
    --capture tests/corpus/reference_export/corpus_capture_01.normalized.json --out /tmp/v1.json
python -m src.protocol.cli verify --rule examples/corpus_rule_v2.json \
    --capture tests/corpus/reference_export/corpus_capture_01.normalized.json --out /tmp/v2.json
python -m src.protocol.cli diff --rule-a examples/corpus_rule_v1.json \
    --rule-b examples/corpus_rule_v2.json --report-a /tmp/v1.json --report-b /tmp/v2.json
```